

LUDO Game Report

231037

September 1, 2024

1. The Structures Used to Represent the Board and the Pieces

In this program, the Ludo game board and pieces are represented using arrays and predefined constants.

Player Colors

The `playerColors[]` array is used to store the names of the colors assigned to the players. This is a string array and it makes it easy to refer to each player by their color. The colors used in this game are: "RED", "GREEN", "YELLOW", and "BLUE".

Path Locations

The `pathLocations[]` array represents the different positions on the Ludo board, including the Base (B), Start (X), and Home (H). This array helps in navigation through the board from the start to the end of the game.

Player Pieces

The `playerPieces[NUM_PLAYERS][NUM_PIECES]` is a 2D array that keeps track of the position of each piece for all players. The value of each element in this array represents the current location of a piece on the board. A value of 0 indicates that a piece is still in the Base (B), 1 means it's at the starting position (X), and 58 indicates it has reached home (H).

Finished Players

The `finishedPlayers[NUM_PLAYERS]` array is used to track which players have finished the game. A value of 1 indicates that the player has finished while 0 means they are still in the game.

Finishing Order

The `finishingOrder[NUM_PLAYERS]` array is used to store the order in which players finish the game. This array helps in displaying the final results at the end of the game.

2. Justification for the Used Structures

Arrays

Arrays were used because they allow for easy indexing and manipulation of data, which is essential for tracking the positions of pieces and the order of players.

Constants (#define)

The use of constants such as `NUM_PLAYERS` and `NUM_PIECES` helps make the code more readable and flexible. If the number of players or pieces changes, these constants can be easily updated without modifying the entire code. This is an effective way to generalize the code.

Data Types

Basic data types such as integers are used to represent the positions of the pieces. This makes it easy to perform arithmetic operations such as moving pieces on the board and checking conditions.

3. Discussion on the Efficiency of the Program with Justification

The program is designed with the following aspects to make it as efficient as possible:

Time Complexity

The most computationally expensive part of the program is the game loop, which is set to iterate until 3 players finish the game. Within each iteration, the program processes moves for one player and checks for valid moves. The complexity here is manageable as it only involves a few arithmetic operations, comparisons, and loops over a small fixed number of players and pieces.

Space Complexity

Since the number of players and pieces is fixed, the program uses a constant amount of predictable and minimal space to store the state of the game.

Efficiency of Roll Calculation

The dice rolling and movement logic is handled efficiently. The program does not perform unnecessary computations and immediately checks for possible moves or outcomes (like capturing another piece) after each roll.

In summary, the program is efficient in terms of both time and space, making it well-suited for the task of simulating a bit-customized LUDO game, which is parallel to the traditional game even on low-end systems. The use of simple structures and straightforward logic ensures that the program is not only effective but also easy to maintain and expand if necessary. The whole output of the game is only about 17KB.

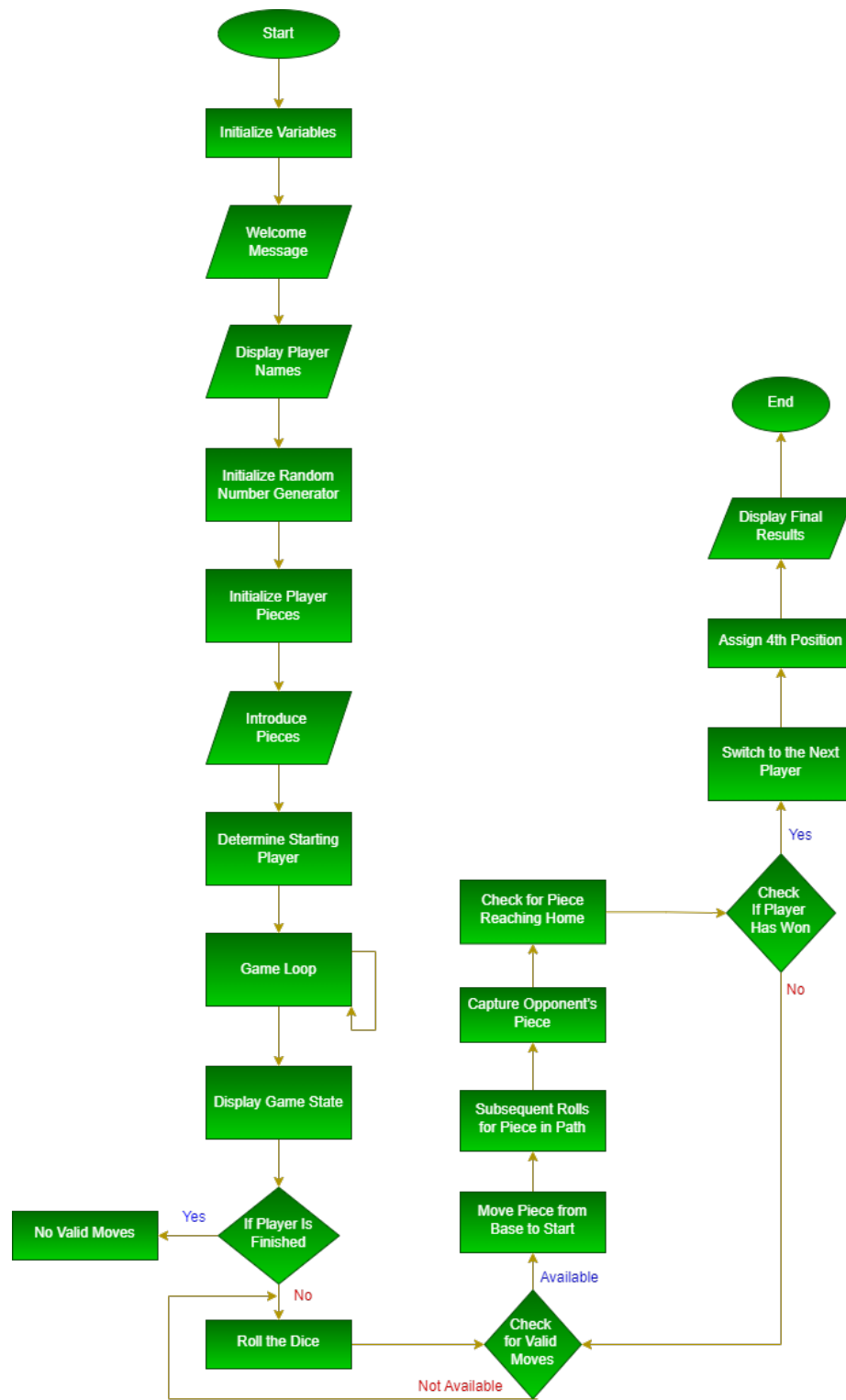


Figure 1: Game flow explained using a Flow Chart